An agent says done. The page says otherwise.

A 10-slide checklist for deciding an agent actually finished the job — not just that the pointer moved.

10 slides · swipe

Anton Petnitsky · Python / agents / backend

A click is an action, not a result.

The failure I see most often: the agent waits, clicks, and closes the ticket as done.

The request returned. The screen changed. The backend did nothing.

The log says: OK.
The user still can't proceed.

Observe → act → assert.

01 Observe the current state before touching anything



02 Take one deliberate action



03 Assert the state changed as intended



04 Only then claim success

No assert step = no finished task.

Anton Petnitsky · Python / agents / backend

Two truth layers. Check both.

An agent reading only the page can be confidently wrong.

LAYER 1 · UI

What the page shows

Field updated?
Dialog closed?
Spinner gone?

LAYER 2 · SYSTEM

What the system did

Backend 200?
Row persisted?
klippy_state: ready?

Green UI + dead backend = false done.

Anton Petnitsky · Python / agents / backend

A green screen can hide a broken process.

I hit this with Klipper: Mainsail loaded fine, nginx returned 200 — and the printer itself had been in error state for hours.

**Check the process,
not the dress code.**

Anton Petnitsky · Python / agents / backend

Never accept 'done' without a named success condition.

01 The field value after submit is the expected one

02 The response status is 2xx and the payload matches the contract

03 The persisted row exists and matches what was sent

04 A reload shows the same state — not just the toast

05 The log line exists and contains the entity ID

06 A second run is idempotent, not a duplicate

Name the condition before you let the agent press the button.

Anton Petnitsky · Python / agents / backend

Proof in code, not in a log line.

```
$ agent.run_task(task)

before = page.locator(field).input_value()

agent.click_submit()

assert page.locator(field).input_value() != before

   assert db.saved(field.id) is not None
```

Read, act, assert — the whole job fits in five lines.

Anton Petnitsky · Python / agents / backend

An agent that cannot refuse is not safe at scale.

No named success condition? Stop, ask, or fail — never guess.

Ambiguity in the spec is the agent's silent failure mode. Surface it, don't smooth it over.

Anton Petnitsky · Python / agents / backend

Minimum viable proof contract.

01 State readable before and after the action

02 A deterministic check, not a human eyeball

03 The check fails loudly — exit code, exception, alert

04 The failure message says WHAT did not change

05 The log lets you replay the decision

If any of these is missing, the task is not done — it is attempted.

Anton Petnitsky · Python / agents / backend

AI AGENTS · KEEP THIS · 10/10

Save this for your next agent postmortem.

Every agent incident I've debugged reduces to one missing check: the one that would have noticed nothing happened.

Save for later review

Anton Petnitsky · Python / agents / backend